

Extension TRIGO pour le compilateur Deca

Implémentation de fonctions mathématiques en simple précision IEEE-754 (32 bits)

Groupe 9 , equipe GL 51

23 Janvier 2026

Résumé

Ce document présente la conception, l'implémentation et la validation de l'extension **TRIGO** pour le compilateur Deca. Cette extension fournit une bibliothèque mathématique complète implémentant les fonctions trigonométriques de base (**sin**, **cos**, **asin**, **acos**, **atan**) ainsi que la fonction utilitaire **ulp** (Unit in the Last Place). L'implémentation cible la machine abstraite IMA et respecte les contraintes de précision de la norme IEEE-754 en simple précision. Nous détaillons les choix algorithmiques fondés sur des approximations polynomiales optimisées (Taylor , Cordic , Interpolation lineaire et polynomiale avec Hermite , ainsi qu'une methode hybride pour les cos ,sin) , les techniques de réduction d'argument pour les fonctions périodiques(cos ,sin), la gestion des cas limites, et la méthodologie de validation rigoureuse(gestion automatique des tests) .

Mots-clés : compilation, calcul scientifique, flottants, trigonométrie, IEEE-754, approximation polynomiale(Hermite,lineaire), validation numérique(des tests), optimisation énergétique.

Table des matières

1	Introduction	4
1.1	Problématique et objectifs de l'extension TRIGO	4
1.1.1	Problématique :	4
1.1.2	Objectifs :	4
1.2	Spécifications imposées	4
2	État de l'art et analyse bibliographique	4
2.1	Représentation des nombres flottants	4
2.1.1	Norme IEEE-754 simple précision	4
2.1.2	Erreurs d'arrondi et ULP	4
2.2	Algorithmes existants pour les fonctions trigonométriques	5
2.2.1	Techniques de réduction d'argument	5
2.2.2	Approximations polynomiales	5
2.3	Méthodes d'évaluation polynomiale	6
2.3.1	Contraintes pour notre machines IMA	7
3	Conception de l'extension	7
3.1	Les Séries de Taylor	7
3.1.1	Principe Mathématique	7
3.1.2	Fonctions Implémentées	7
3.1.3	Gestion du Domaine et Précision	7
3.1.4	Exemple de Code Deca	8
3.2	Algorithme Cordic :Mode Rotation	8
3.2.1	Principe de l'Algorithme :Mode Rotation	8

3.2.2	Équations de Récurrence	8
3.2.3	Table de Constantes	9
3.2.4	Optimisation pour la langage Deca	9
3.2.5	Avantages par rapport aux Séries de Taylor	9
3.2.6	Disavantages D'algorithme CORDIC	9
3.2.7	Exemple de boucle CORDIC en Deca	9
3.3	Algorithme Cordic :Mode Vectoriel	10
3.3.1	Principe de L'algorithme en mode vectoriel	10
3.3.2	les points forts de cet algorithme	10
3.3.3	Les Limites de cet Algorithme, et comment on les gerent	10
3.3.4	Performance	10
3.4	Algorithme de Table Pré-calculées	11
3.4.1	Étape 1 : Préparation des donnees (une fois pour toutes)	11
3.4.2	Étape 2 : Pour calculer $\sin(x)$ pour x quelconque	11
3.4.3	Le Probleme en en Deca	11
3.4.4	Une solution	11
3.4.5	Algorithme de recherche :	12
4	Conception de la classe Math	12
4.1	Structure de la classe Math	12
4.2	Choix de conception majeurs	13
4.2.1	Précision vs performance	13
4.2.2	Gestion des cas particuliers	13
4.2.3	Stratégie de réduction d'argument	13
4.3	Organisation des fichiers	14
5	Algorithmes et méthodes mathématiques	15
5.1	Fonction $\text{ulp}(\text{float } x)$	15
5.1.1	Définition mathématique	15
5.1.2	Algorithme d'implémentation	15
5.2	Fonctions $\sin$ et $\cos$	16
5.2.1	Implémentation de la fonction : $\sin$	16
5.2.2	Réduction d'argument : Principe déviser pour régner	17
5.2.3	Algorithme Cordic en mode Rotation pour $\cos$ et $\sin$	17
5.2.4	Implémentation des tables pré-calculées de $\cos$ et $\sin$	18
5.2.5	Implémentation de la fonction $\cos$	19
5.3	Fonction atan	20
5.3.1	Implementation de la fonction atan	20
5.3.2	Réduction de l'argument pour la fonction Atan	20
5.4	Fonction asin	21
5.5	Fonction acos	21
6	Validation et tests	21
6.1	Notre Stratégie de validation	21
6.2	Genre de tests	22
6.3	Référence de précision et Visualisation des erreurs	22
6.4	Tests de performance	26
6.4.1	Temps d'exécution	26

7	Limitations et perspectives	27
7.1	Limitations actuelles	27
7.1.1	Précision	27
7.1.2	Performance	27
7.2	Améliorations possibles	27
7.2.1	Algorithmiques	27
7.3	Des Extensions possibles pour le futur	27
8	Conclusion	27
8.1	Récapitulatif des contributions	27
	Références	28

1 Introduction

Le projet Génie Logiciel consiste à développer un compilateur complet pour le langage Deca, un langage orienté objet inspiré de Java (avec des restrictions) .

Ce projet intègre plusieurs dimensions : génie logiciel, compilation, travail en équipe(5 personnes de différentes visions et idéologies) et développement de qualité.

L'extension TRIGO s'inscrit dans la partie *extension* du projet, permettant à nous les étudiants ingénieurs d'approfondir un domaine spécifique tout en respectant des contraintes techniques strictes(une documentation floue , et pousse à faire des choix importantes) .

1.1 Problématique et objectifs de l'extension TRIGO

1.1.1 Problématique :

Comment fournir des fonctions trigonométriques précises (notion de précision des flottants) et efficaces (notion d'énergie et environnement) dans un langage compilé vers une machine virtuelle "ima" simple ?

1.1.2 Objectifs :

1. Implémenter `sin`, `cos`, `asin`, `atan`, `ulp` en Deca/assembleur IMA .
et on a ajouté aussi `acos` , avec des méthodes ou bien des fonctions auxiliaires comme la `sqrt(x>0)` , `abs(x)`, `Cordic_cos` , `Cordic_sin` et d'autres fonctions .
2. Atteindre une précision maximale permise par le format IEEE-754 simple précision(<2ulp qui est en pratique et aussi en théorie impossible à atteindre pour tous les entrees.)
3. Optimiser la vitesse d'exécution et la consommation mémoire (Cache , en raison de l'utilisation des valeurs pré-calculée pour `sin` , `cos` et `Cordic` , (`atan`)
4. Analyser l'impact énergétique des choix d'implémentation

1.2 Spécifications imposées

- Remplissage du Fichier `Math.decah` dans la bibliothèque standard
- Méthodes requises : `ulp(float)`, `sin(float)`, `cos(float)`, `asin(float)`, `atan(float)` et `acos(float)` , avec possibilité d'ajouter des méthodes auxiliaires avec préfixe `_nomMethode()`
- Gestion des erreurs : `asin` avec argument hors `[-1,1]` provoque une erreur d'exécution
- Gestion des cas limites : de `Infinite` et `NaN`(implicit via la vérification contextuelle du compilateur et aussi de `ima`)

2 État de l'art et analyse bibliographique

2.1 Représentation des nombres flottants

2.1.1 Norme IEEE-754 simple précision

Le sous langage deca , accepte des flottants en Format 32 bits : c'est à-dire : 1 bit de signe, 8 bits d'exposant, 23 bits de mantisse.

Précision relative : 2^{-23} vaut d'environ 1.2×10^{-7} .

2.1.2 Erreurs d'arrondi et ULP

L'ULP (Unit in the Last Place) représente la distance entre deux flottants consécutifs.

Pour $x \in \mathbb{F}$ l'espace des flottants 32bits en simple précision, `ulp(x)` est la valeur du bit de poids le plus faible de sa mantisse.

2.2 Algorithmes existants pour les fonctions trigonométriques

2.2.1 Techniques de réduction d'argument

- **Réduction modulaire naïve** : perte de précision pour les grands arguments
- **Méthode de Payne et Hanek : réduction exacte pour grands arguments** (Payne-Hanek réduit les grands angles pour sin/cos en évitant les erreurs. Au lieu d'utiliser π comme nombre float à virgule (trop imprécis), il traite π comme une fraction exacte en mémoire (il stock la vraie valeur de π avec le nombre de chiffre qu'il veut). Ça permet de calculer $\sin(10^{20})$ aussi précisément que $\sin(0.1)$. Mais, comme chaque algorithme a ses points forts et points de faibles, la méthode de Payne et Hanek présente une implémentation complexe (car c'est difficile à coder correctement) à cause de la diversité des cas limites, en plus la gestion multi-précision qui est lourde. Aussi, la performance est coûteuse car pour les grands arguments, c'est 10 - 100x plus lent que le calcul normal (opérations sur grands entiers). En plus, un autre défaut est que, pour des très grands nombres (par exemple $> 10^{100}$), et des valeurs dénormalisées, la méthode nécessite une gestion spécifique délicate pour chaque cas particuliers).
- **Table de pré-calcul** pour multiples de $\pi/2$ (on l'a implémenté)

2.2.2 Approximations polynomiales

- **Développement de Taylor** : précision médiocre (et perte totale de la précision) loin du point d'expansion $x > 0.1$ pour sin, cos.
- **Polynômes de Chebyshev** : Ils répartissent l'erreur d'approximation de façon uniforme sur l'intervalle, contrairement à Taylor qui est très précis au centre mais diverge aux bords. On utilise leurs racines comme points d'interpolation pour minimiser l'erreur maximale.

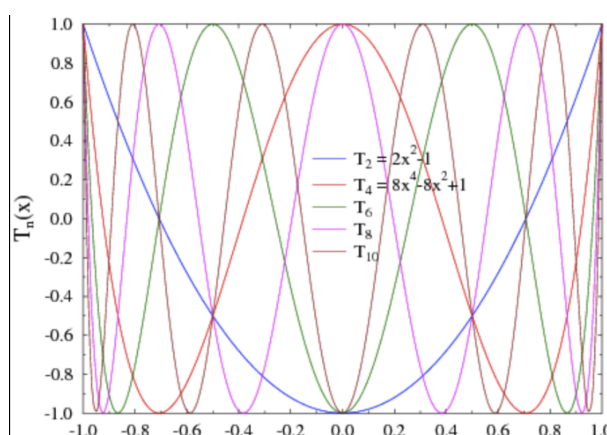


FIGURE 1 – Les premiers polynômes de Chebychev

pourquoi je les ai pas utilisés ?

En fait, le calcul des coefficients est parfois complexe et nécessite une préanalyse numérique pour chaque fonction (sin, cos, exp ..).

Aussi, l'évaluation est parfois plus coûteuse, même si moins de termes, la structure est moins régulière que les polynômes de Taylor (donc il faut gérer plusieurs cas particuliers).

- **Approximations de minimax** (algorithme de Remez) : Algorithme itératif qui produit le polynôme de degré donné le plus proche d'une fonction sur un intervalle, au sens de l'erreur maximale (qu'il faut minimiser).

```

1.07586... at source line 1
Test: asin_0.88
  Obtenu   : 1.07586e+00
  Attendu  : 1.075862e+00
1.075862e+00
  ULP      : 5.96046e-08
  Erreur relative : ULP
  OK (<= 2 ULP)

Test: asin_0.9
  Obtenu   : 1.11977e+00
  Attendu  : 1.119769e+00
  ULP      : 5.96046e-08
  Erreur relative : 16.777229 ULP
  un peu mal (>4 ULP et <100 ULP)

Test: asin_0.9659258
  Obtenu   : 1.30900e+00
  Attendu  : 1.308997e+00
  ULP      : 5.96046e-08
  Erreur relative : 50.331686 ULP
  un peu mal (>4 ULP et <100 ULP)

Test: asin_0.99
  Obtenu   : 1.42926e+00
  Attendu  : 1.429257e+00
  ULP      : 5.96046e-08
  Erreur relative : 50.331686 ULP
  un peu mal (>4 ULP et <100 ULP)

```

FIGURE 2 – Algo de Remez : on approche f par P , en minimisant l'erreur maximal

Pourquoi je l'ai pas utilisé ? car il est lourd à mettre en œuvre (on a besoin d'un solveur numérique externe comme Sollya), en plus les coefficients ne sont pas "jolis" (des flottants arbitraires), donc moins prévisibles en précision finie (32bits), en fin, certes l'erreur est uniforme (presque ne dépend pas de l'entrée), mais parfois on préfère une erreur variable selon l'entrée, par exemple pour la fonction sin, on doit pas faire des erreurs pour $x \rightarrow 0$, par contre on peut admettre des erreurs assez grandes pour $x > 1000$.

- **Interpolation de Lagrange et Hermite** (faites en module de **Modélisation Géométrique** et aussi en lien avec le module de 1ère année Ensimag **Méthode numérique**. pour ces deux algorithmes, il en faut trop de points pour une haute précision sur tout R .

2.3 Méthodes d'évaluation polynomiale

- **Schéma de Horner** : $P(x) = a_0 + x(a_1 + x(a_2 + \dots))$ - stable et efficace, mais ne réduit pas l'erreur d'approximation (erreur qu'on fait en approchant f par Polynôme P), mais juste l'erreur d'évaluation (erreur de calcul float*float ou x^i).
- **Évaluation en double précision intermédiaire** pour limiter les erreurs (**impossible en Deca!**)
- **Utilisation de FMA** (Fused Multiply-Add) : $a \times b + c$ avec un seul arrondi (avec l'algorithme Cordic, on n'a pas besoin, car on fait pas de mult, juste des additions et

soustractions)

2.3.1 Contraintes pour notre machines IMA

- Besoin d'Algorithmes adaptés aux contraintes de la machine IMA (registres limités ,en general 16 registres)
- Compromis entre précision, vitesse et taille du code(respet d'environnement , et cout energetique)
- Techniques de programmation par table pré-calculés

3 Conception de l'extension

3.1 Les Séries de Taylor

3.1.1 Principe Mathématique

Pour implémenter les fonctions trigonométriques (surout dans des cas limites comme $x \rightarrow 0$) , nous utilisons les développements en série de Taylor pour gérer des cas particuliers des entrees , là ou les séries de taylor convergent rapidement et de façon précise .

La série de Taylor d'une fonction f infiniment dérivable au voisinage d'un point a est :

$$f(x) = \sum_{n=0}^{\infty} \frac{f^{(n)}(a)}{n!} (x - a)^n$$

Pour notre compilateur, nous utilisons les développements limités au voisinage de 0 (par exemple pour la fonction **sin** et meme **atan** , **asin** , pour une entree $x < 0.001$,on utilise une serie de taylor à l'ordre 1 ou 3 (selon la fonction et la precision qu'on cherche) .

3.1.2 Fonctions Implémentées

Fonction Sinus Le développement utilisé pour $\sin(x)$ et pour $0 < x < 0.001$ est :

$$\sin(x) = x$$

Fonction Atan Le développement utilisé pour $\arctan(x)$ et pour $x < 0.151$ est :

$$\text{Atan}(x) = x - \frac{x^3}{3} + \frac{x^5}{5} - \frac{x^7}{7}$$

L'analyse des erreurs d'arrondi pour la série de Taylor tronquée à l'ordre 7 montre que l'erreur absolue $E_T(x)$ satisfait :

$$E_T(x) \leq \frac{|x|^9}{9} \quad \text{pour } |x| < 0.151$$

Cette borne garantit une erreur inférieure à $10^{-7} \approx 2^{-23}$ dans l'intervalle considéré. "un bon compomis entre précision et aussi une optimisation énergétique (environnement)"

la même procedure pour tous les fonctions avec laquelle on a utilisé cette implementation de série de Taylor

3.1.3 Gestion du Domaine et Précision

Comme l'approximation de Taylor et aussi les autres algorithmes qu'on vient d'utiliser perdent en précision lorsque $|x|$ s'éloigne de 0 , ou si x prend des valeurs extremment gros comme 10000 pour les fonctions sin et cos .

Nous avons donc implémenté :

1. **Réduction d'intervalle** : Utilisation de la périodicité 2π et des propriétés de symétrie pour ramener x dans l'intervalle $[0, 2\pi]$.
2. **gestion des valeurs négatives** : utilisation de la symétrie de cos ,sin et atan... pour ne traiter que les valeurs positives et à la fin conclure selon le signe

3.1.4 Exemple de Code Deca

```
float sin(float x) {
    if (x_red < TRES_PETIT) {
        result = x_red;           // sin(x) = x
    }
    else if{....}
}
```

Dans notre implementation , $TREE_PETIT = 0.001$, ce qui permet aux series de taylor de converger rapidement et efficacement sans perte de precision.

=====

3.2 Algorithme Cordic :Mode Rotation

3.2.1 Principe de l'Algorithme :Mode Rotation

L'algorithme CORDIC en mode de rotation permet de calculer le sinus et le cosinus d'un angle θ en effectuant une série de rotations successives d'un vecteur initial $v_0 = (K, 0)$ avec la constante K correspond au produit inverse des cosinus des angles élémentaires θ_i qu'on vient de stocker dans la table Cordic et qu'on les utilise à chaque itération. dans notre cas : ($K = 0.60725293500888144482f$).

À chaque étape i , la longueur du vecteur est multipliée par $\sqrt{1 + 2^{-2i}}$.

Après n itérations (pour notre cas le $n=24$) ,Le choix $n = 24$ itérations pour CORDIC résulte d'une analyse d'erreur théorique :

l'erreur angulaire résiduelle E_θ après n itérations vérifie $E_\theta \leq \alpha_n \approx 2^{-n}$ rad.

Pour $n = 24$, $E_\theta < 2^{-24} \approx 6 \times 10^{-8}$ rad, ce qui est comparable à la précision machine en simple précision (32 bits et 23 en mantisse et qui vaut $2^{-23} = 1.192 \times 10^{-7}$) .

Au lieu d'effectuer une rotation directe de l'angle θ , on effectue n micro-rotations d'angles élémentaires α_i tels que $\tan(\alpha_i) = 2^{-i}$.

Pour ce faire , on stock dans un fichier CordicTable.deca une méthode qui permet de faire une **recherche binaire** intelligentes (l'algorithme présente la méthode de recherche dichotomique optimisée pour l'accès aux tables de précalcul.

La complexité est $O(\log_2 n)$ avec $n = 24$ entrées, garantissant un accès en 5 comparaisons maximum). et donc de retourner le bon **atan**(2^{-i} au bon moment et au bon endroit.

3.2.2 Équations de Récurrence

À chaque itération i , on met à jour les coordonnées (x, y) du vecteur et l'angle restant z :

$$\begin{cases} x_{i+1} = x_i - \sigma_i \cdot y_i \cdot 2^{-i} \\ y_{i+1} = y_i + \sigma_i \cdot x_i \cdot 2^{-i} \\ z_{i+1} = z_i - \sigma_i \cdot \alpha_i \end{cases}$$

Où $\sigma_i = 1$ si $z_i \geq 0$, et $\sigma_i = -1$ sinon.

Après n itérations, les coordonnées sont proportionnelles au cosinus et au sinus :

$$x_n \approx \cos(\theta), \quad y_n \approx \sin(\theta)$$

3.2.3 Table de Constantes

Pour fonctionner, l'algorithme nécessite une table des angles $\alpha_i = \arctan(2^{-i})$. Ces valeurs sont stockées en un fichier `CordicTable.deca`, contenant une classe avec une methode qui retourne le bon $\text{pow2i}=2^{-i}$ et une autre methode `_getCordic(i)` qui retourne le bon $\text{Atan}(2^{-i})$ lors de l'initialisation de la classe `Math`.

3.2.4 Optimisation pour la langage Deca

bien que le langage deca n'offre pas des tableaux, ni de boucle `for`, on a mis des optimisations pour contourner ces problemes

- utilisation de **while** au lieu de `for`
- Utilisation des méthodes de **Recherche Binaire** pour trouver le plus vite possible la valeur adequate que se soit pour `pow2i` ou pour les tables Cordic, et meme pour les Tables de Cos et Sin, qu'on va aborder dans la prochaine section

3.2.5 Avantages par rapport aux Séries de Taylor

- **Convergence uniforme** : Contrairement à Taylor, la précision est la même sur tout l'intervalle $[-\pi/2, \pi/2]$.
- **Complexité fixe** : Le temps d'exécution est constant (exactement n itérations), ce qui facilite le calcul du temps d'exécution (WCET).
- **Efficacité matérielle** : Pas besoin de multiplications complexes pour les puissances de x .

3.2.6 Disavantages D'algorithme CORDIC

L'algorithme Cordic en mode Rotation necessite une reduction d'intervalles en $[-2\pi, 2\pi]$, aussi il parait juste pour les valeurs entre $[-\pi/2, \pi/2]$, pour les autres entrees, il fait le calcul, mais avec beaucoup d'erreurs surtout en 32bits. C'est pour cette raison, qu'on a essayer de combiner les avantages de Cordic avec celle de Taylor et aussi le 3eme algo (Tables Pré-calculées). "Methpde Hybride"

3.2.7 Exemple de boucle CORDIC en Deca

// Extrait de l'implémentation itérative de cordic pour calculer sin:

```
i=0;
while(i < iterations) {
    xOld = x;
    pow2i = table._getPow2(i);

    if (z >= 0) {
        x = x - y * pow2i;
        y = y + xOld * pow2i;
        z = z - table._getCordic(i);
    } else {
        x = x + y * pow2i;
        y = y - xOld * pow2i;
        z = z + table._getCordic(i);
    }
    i = i+1;
}
```

3.3 Algorithme CORDIC : Mode Vectoriel

3.3.1 Principe de l'algorithme en mode vectoriel

L'idée de cet algorithme est de ramener un vecteur (x, y) sur l'axe des x par rotations successives d'angles prédéfinis $\theta_i = \arctan(2^{-i})$.

Relations de récurrence

Pour $i = 0, 1, \dots, n-1$:

$$\begin{cases} x_{i+1} = x_i - d_i \cdot y_i \cdot 2^{-i} \\ y_{i+1} = y_i + d_i \cdot x_i \cdot 2^{-i} \\ z_{i+1} = z_i - d_i \cdot \alpha_i \end{cases}$$

avec $d_i = \begin{cases} -1 & \text{si } y_i \geq 0 \\ +1 & \text{si } y_i < 0 \end{cases}$ et $\alpha_i = \arctan(2^{-i})$.

Après n itérations :

$$\begin{cases} x_n \approx K \cdot \sqrt{x_0^2 + y_0^2} \\ y_n \approx 0 \\ z_n \approx \arctan\left(\frac{y_0}{x_0}\right) = \theta \end{cases}$$

où $K = \prod_{i=0}^{n-1} \frac{1}{\sqrt{1+2^{-2i}}} = 0.60725293500888144482$.

Application pour $\arctan(f)$

Pour calculer $\arctan(f)$ avec $f \in [-1, 1]$:

Initialisation : $x_0 = 1, \quad y_0 = f, \quad z_0 = 0$

Après CORDIC : $\arctan(f) \approx z_n$.

3.3.2 les points forts de cet algorithme

- **Additions/soustractions seulement** : pas de multiplications coûteuses
- **Convergence garantie** : pour $|t| \leq 1$, erreur $\sim 2^{-n}$, en plus nous utilisons cet algorithme juste pour les valeurs de $f \in [0.151, 1]$, dans cette intervalle les performances de Cordic sont les meilleurs
- **Table petite** : $\alpha_i = \arctan(2^{-i})$ pour $i = 0..n-1$, avec $n=30$

3.3.3 Les Limites de cet Algorithme, et comment on les gère

- **Intervalle limité** : nécessite $|t| \leq 1$ (ou $t = \tan \theta$ avec $|\theta| \leq \pi/2$) $\rightarrow$ réduction en $[-1, 1]$ avec $\arctan(1/x) = \pi/2 - \arctan(x)$
- **Erreur d'arrondi** : accumulation sur n itérations ($\sim n$ ULP) $\rightarrow$ on prend $n=30$ (bon compromis)

3.3.4 Performance

- Précision : $\sim 2^{-n}$ radians
 - Complexité : n itérations, $3n$ additions
 - Mémoire : table de n constantes θ_i
-

3.4 Algorithme de Table Pré-calculées

Au lieu de calculer $\sin(x)$, $\cos(x)$... à la volée, on l'a calculé à l'avance pour plein de valeurs, et on se débrouille pour x entre ces valeurs avec des méthodes d'interpolation (linéaire, Lagrange, Hermite ...)

3.4.1 Étape 1 : Préparation des données (une fois pour toutes)

On calcule une fois et on stocke : $\sin(0) = 0.000000$ $\sin(\pi/255) = ???$ $\sin(2/255) = ???$ $\sin(3/255) = ???$... $\sin(255/255) = ???$ Dans ce cas, on a choisi d'initialiser avec 256 valeurs (on peut choisir 1024 valeurs, ou juste 512)

3.4.2 Étape 2 : Pour calculer $\sin(x)$ pour x quelconque

- Il faut réduire x en $[0, 2\pi]$ (car $\sin$, $\cos$ sont périodiques)
- Trouver les deux valeurs de la table les plus proches
- Estimer $\sin(x)$ par interpolation entre ces deux valeurs

3.4.3 Le Problème en en Deca

- Pas de tableaux : comment stocker 256 valeurs
- et si on écrit 256 if/else : code énorme, et lent

3.4.4 Une solution

- Une analyse théorique pour les float 32, qui ont 6 chiffres significatifs, montre qu'on peut avoir des bons résultats (avec un compromis efficacité/énergie) en se limitant des tables de 128 valeurs pré-calculées, au lieu de 256 valeurs.
- Une table sous forme des if/else, avec une dichotomie
- pour une valeur de i , la recherche binaire distingue entre les deux cas $i > 64$ et $i < 64$, et si $i > 64$, on distingue entre $i > 32$ ou non ...
- **cette méthode, permet de passer d'une complexité de 128 en pire des cas, vers $\log(256) = 7$ en pire des cas!**, donc on gagne environ x18 plus de temps de comparaison ($128/7 = 18$)

3.4.5 Algorithme de recherche :

Algorithm 1 Recherche par dichotomie

```
1: if  $i < 64$  then
2:   if  $i < 32$  then
3:     if  $i < 16$  then
4:       if  $i < 8$  then
5:         if  $i < 4$  then
6:           if  $i < 2$  then
7:             if  $i = 0$  then return  $v_0$ 
8:             elsereturn  $v_1$ 
9:             end if
10:          else
11:            if  $i = 2$  then return  $v_2$ 
12:            elsereturn  $v_3$ 
13:            end if
14:          end if
15:        else
16:          .....
17:
18:          if  $i = 6$  then return  $v_6$ 
19:          elsereturn  $v_7$ 
20:          end if
21:        end if
22:      end if
23:    else
24:      ..
```

Cet algorithme est appliquée à la fois pour la table cos et sin , (aussi pour CordicTable)

=====

=====

4 Conception de la classe Math

4.1 Structure de la classe Math

```
class Math {
  // Methodes publiques (API utilisateur)
  float ulp(float f);
  float sin(float f);
  float cos(float f);
  float asin(float f);
  float acos(float f);
  float atan(float f);

  // Methodes internes (auxiliares)
  float _tableSin(float x);
  float _tableCos(float x);
  float _abs(float x);
  float _sqrt(float x >0);
  float _sqrt1(float x >0); //2eme version de sqrt
```

```

// constantes de base
protected float PI = 3.141592653589793f;
protected float TWO_PI = 6.283185307179586f;
protected float PI_OVER_2 = 1.5707963267948966f;
protected float PI_OVER_4 = 0.78539816339744830962f;
protected float CORDIC_K_CONST = 0.60725293500888144482f;

//des Constantes pour la table SIN , COS:
protected int CACHE_SIZE = 512;
protected float CACHE_STEP = PI_OVER_2/(CACHE_SIZE -1);
protected int CORDIC_TABLE_SIZE = 32;

// Seuils pour choix d'algorithme (compromis pr cision/energie)
protected float TRES_PETIT = 0.001f;
protected float PETIT = 0.1f; // Pour table Cos Sin
protected float VOISIN_PI_2 = 1.472f;
}

```

4.2 Choix de conception majeurs

4.2.1 Précision vs performance

Nous avons opté pour une approche équilibrée : Erreur maximale cible : < 2 ULP pour sin/cos (pour la majorite des cas , et de ne pas deppasser une dizaine de ulp pour les cas extremes , là ou la precision diminue , par exemple cos(100000) ou bien sin(0.0000001)), et de meme < 2 ULP pour asin/atan/acos

pour la fonction ulp , notre objectif est de retourner la valeurs la plus proches possible de celle retourner par java.lang.Math.ulp(x)

car cette méthode ulp est la base des tests pour toute l'extension

4.2.2 Gestion des cas particuliers

Fonction	Cas particulier	Traitement
sin/cos	± 0	Retourner ± 0 (sin) ou 1 (cos)
asin/acos	$ x > 1$	Erreur (overflow)
asin/acos	$ x = 1$	Retourner $\pm \pi/2$ ou 0
atan	± 0	Retourner ± 0

TABLE 1 – Gestion des cas particuliers

4.2.3 Stratégie de réduction d'argument

Pour les fonctions périodiques (sin, cos), nous utilisons :

1. Réduction en $[0, 2\pi]$ (suivi d'une autre reduction vers le bon quadrant et une gestion fine du signe)
2. pour les très grands arguments : on utilise une méthode fondée sur la recherche binaire(dichotomie) , pour trouver la valeur stockée la plus proche (une liste des valeurs multiples (en puissance) de 2π (a savoir 2π , $2^2\pi$, $2^3\pi$, $2^4\pi$, $2^5\pi$ $2^{14}\pi$).

Le choix de cette conception est fondée sur l'idee de **diviser pour régner** populaire en recherche binaire

Algorithm 2 Reduction d'argument pour sin/cos

```
1: procedure _reduction2PIRecursive(f)
2:   //cas de base
3:   if f > 0.0 et f < TWO_PI then return x
4:   end if
5:   if f < 0.0 then return _reduction2PIRecursive(f + TWO_PI)
6:   end if
7:   // x >=  $2^{14}\pi$ 
8:   if f >= 51471.85403641517 then return _reduction2PIRecursive(x -
51471.85403641517 f)
9:   end if
10:  // x >=  $2^{13}\pi$ 
11:  if f >= 25735.927018207584 then return _reduction2PIRecursive(x -
25735.927018207584 f)
12:  end if
13:  .....
14:  // x >=  $2^1\pi$ 
15:  if f >= TWO_PI then return _reduction2PIRecursive(x - TWO_PI)
16:  end if
[1]
```

4.3 Organisation des fichiers

- `src/main/resources/include/Math.decah` : fichier principal
- `src/test/deca/Trigo/(in)valid` : tests de validation (valid et invalid)
- `src/test/deca/Trigo/*.sh` : des scripts .sh pour creer des tests , les supprimer et de lancer la compilation sur l'ensemble des tests valid/invalid
- **Remarque** : invalid dans ce contexte signifie que le test en question doit echoue ,(par exemple `cos(Infinity)` (erreur en codegen) , `sin(1.0/0.0)` (division par zero) , aussi `asin(3.0)` , `acos(-2.8)` car le codomaine de `asin` et `acos` est `[-1,1]`)
- `src/test/deca/Trigo/*.java` ce genre des fichiers , est la base des tests , c'est par le biais du code java qu'on a genere une liste des valeurs qu'on test avec le script .sh
- `src/test/deca/Trigo/*.txt` l'execution de code java pour `TestCos.java` ou `TestAcosAsin.java` par exemple donne une structure text avec le format :
nom_fichier;nom_fonction;valeur;resultat_attendu le resultat attendu est soit "erreur" (donc on met le test dans `./invalid` , sinon ,on le met dans `./valid`)
- `src/test/deca/Trigo/*.py` un fichier python3 est aussi utilisé pour calculer et remplir `CordicTable.deca` , contenant les deux methodes `pow2i` et `Atan(2-i)` qu'on vient d'en parler dans la partie Cordic
- `src/test/deca/Context/Trigo/*.deca` une liste des tests valid ou invalid contextuellement contenant des appellees a la bibliotheque Math
- `src/test/deca/synt/Trigo/*.deca` une liste des tests valid ou invalid syntaxiquement contenant des appellees a la bibliotheque Math
- `src/test/deca/Trigo/trigo*.deca` des tests pour la demonstration **Remarque** : on n'a pas mis des tests de Trigo dans codegen , car ce genre de tests est deja mis dans `src/test/deca/Trigo/(in)valid`

=====

5 Algorithmes et méthodes mathématiques

5.1 Fonction ulp(float x)

5.1.1 Définition mathématique

Pour $x \in \mathbb{F}$ (ensemble des flottants normalisés) :

$$\text{ulp}(x) = 2^{\lfloor \log_2 |x| \rfloor - 23}$$

5.1.2 Algorithme d'implémentation

Algorithm 3 Calcul de ulp(x) en simple précision

```
1: procedure ULP(f)
2:   // Cas spéciaux IEEE-754
3:   if f = 0.0 then return  $2^{-149}$ 
4:   end if
5:   // Valeur absolue
6:   if f < 0.0 then val  $\leftarrow -f$ 
7:   else val  $\leftarrow f$ 
8:   end if
9:   // Recherche exposant exp tel que  $2^{\text{exp}} \leq \text{val} < 2^{\text{exp}+1}$  exp  $\leftarrow 0$ 
10:  if val  $\geq 1.0$  then
11:    while val  $\geq 2.0$  do val  $\leftarrow \text{val}/2.0$ ; exp  $\leftarrow \text{exp} + 1$ 
12:    end while
13:  else
14:    while val < 1.0 do val  $\leftarrow \text{val} \times 2.0$ ; exp  $\leftarrow \text{exp} - 1$ 
15:    end while
16:  end if
17:  // Calcul ULP =  $2^{\text{exp}-23}$  targetExp  $\leftarrow \text{exp} - 23$ 
18:  if targetExp < -126 then return  $2^{-149}$ 
19:  end if
20:  // Calcul  $2^{\text{targetExp}}$  par multiplications/divisions successives res  $\leftarrow 1.0$ 
21:  if targetExp  $\geq 0$  then
22:    for i = 0 to targetExp - 1 do res  $\leftarrow \text{res} \times 2.0$ 
23:    end for
24:  else targetExp  $\leftarrow -\text{targetExp}$ 
25:    for i = 0 to targetExp - 1 do res  $\leftarrow \text{res}/2.0$ 
26:    end for
27:  end if
28:  return res
29: end procedure==0
```

=====

5.2 Fonctions sin et cos

5.2.1 Implémentation de la fonction : sin

Algorithm 4 Calcul de $\sin(x)$ en simple précision

```

1: procédure SIN( $f$ )
2:   // Gestion du signe
    $negative \leftarrow false$ 
3:   if  $f < 0$  then
4:      $negative \leftarrow true$ 
5:      $f \leftarrow -f$ 
6:   end if
7:   // Réduction modulo  $2\pi$ 
    $x\_red \leftarrow \_reduction2PIRecursive(f)$ 
8:   // Réduction au premier demi-cercle  $[0, \pi]$ 
9:   if  $x\_red > \pi$  then
10:     $x\_red \leftarrow x\_red - \pi$ 
11:     $negative \leftarrow true$ 
12:   end if
13:   // Réduction au premier quadrant  $[0, \pi/2]$ 
14:   if  $x\_red > \pi/2$  then
15:     $x\_red \leftarrow \pi/2 - x\_red$ 
16:   end if
17:   // Choix de la méthode selon la magnitude
18:   if  $x\_red < TRES\_PETIT = 0.001$  then
19:     $result \leftarrow x\_red$ 
20:   else if  $x\_red < PETIT = 0.1$  then
21:     $result \leftarrow \_tableSin(x\_red)$ 
22:   else if  $x\_red > VOISIN\_PI\_2 = 1.471$  then
23:     $res \leftarrow \pi/2 - x\_red$ 
24:    if  $res < TRES\_PETIT$  then
25:       $result \leftarrow 1.0$ 
26:    else if  $res < PETIT$  then
27:       $result \leftarrow \_tableCos(res)$ 
28:    else
29:       $result \leftarrow 1.0$ 
30:    end if
31:   else
32:     $result \leftarrow \_cordicSin(x\_red)$ 
33:   end if
34:   // Application du signe
35:   if  $negative$  then
36:     return  $-result$ 
37:   else
38:     return  $result$ 
39:   end if
40: end procédure

```

▷ Dans $[\pi, 2\pi]$, sin est négatif

▷ $\sin(\pi - x) = \sin(x)$

▷ $\sin(x) \approx x$

▷ Table d'interpolation

▷ $\cos(res) \approx 1$

▷ $\cos(res)$ par table

▷ Algorithme CORDIC

5.2.2 Réduction d'argument : Principe déviser pour régner

Algorithm 5 Réduction modulo 2π récursive

```

1: procedure _REDUCTION2PIRECURSIVE( $x$ )
2:   if  $0 \leq x < 2\pi$  then                                     ▷ Cas de base
3:     return  $x$ 
4:   end if
5:   if  $x < 0$  then                                             ▷ Cas négatif
6:     return _reduction2PIRecursive( $x + 2\pi$ )
7:   end if
8:   // Réduction avec multiples de  $\pi$  pré-calculés
9:   if  $x \geq 16384\pi$  then return _reduction2PIRecursive( $x - 16384\pi$ )
10:  end if
11:  if  $x \geq 8192\pi$  then return _reduction2PIRecursive( $x - 8192\pi$ )
12:  end if
13:  if  $x \geq 4096\pi$  then return _reduction2PIRecursive( $x - 4096\pi$ )
14:  end if
15:  if  $x \geq 2048\pi$  then return _reduction2PIRecursive( $x - 2048\pi$ )
16:  end if
17:  if  $x \geq 1024\pi$  then return _reduction2PIRecursive( $x - 1024\pi$ )
18:  end if
19:  if  $x \geq 512\pi$  then return _reduction2PIRecursive( $x - 512\pi$ )
20:  end if
21:  if  $x \geq 256\pi$  then return _reduction2PIRecursive( $x - 256\pi$ )
22:  end if
23:  if  $x \geq 128\pi$  then return _reduction2PIRecursive( $x - 128\pi$ )
24:  end if
25:  if  $x \geq 64\pi$  then return _reduction2PIRecursive( $x - 64\pi$ )
26:  end if
27:  if  $x \geq 32\pi$  then return _reduction2PIRecursive( $x - 32\pi$ )
28:  end if
29:  if  $x \geq 16\pi$  then return _reduction2PIRecursive( $x - 16\pi$ )
30:  end if
31:  if  $x \geq 8\pi$  then return _reduction2PIRecursive( $x - 8\pi$ )
32:  end if
33:  if  $x \geq 4\pi$  then return _reduction2PIRecursive( $x - 4\pi$ )
34:  end if
35:  // Dernière réduction
36:  return _reduction2PIRecursive( $x - 2\pi$ )
37: end procedure

```

5.2.3 Algorithme CORDIC en mode Rotation pour cos et sin

en donne à titre d'exemple l'implémentation de `_cordicSin`, la méthode `_cordicCos` est similaire à quelque différences fines.

Algorithm 6 CORDIC pour $\sin(x)$

```
1: procedure _CORDICSIN(angle)
2:   // Réduction au premier octant  $[0, \pi/4]$ 
3:   CordicTable table = new CordicTable();
4:   //instantie l'objet table de la classe CordicTable (contenant les deux methodes
   _getCordic(i) qui return  $\arctan(2^{-i})$  et _getPow2(i) qui return le  $2^{-i}$ 
   inverse  $\leftarrow$  false
5:   if angle >  $\pi/4$  then
6:     angle  $\leftarrow \pi/2 - \textit{angle}$ 
7:     inverse  $\leftarrow$  true
8:   end if
9:   // Initialisation CORDIC
   x  $\leftarrow K$  //  $K = 0.607252935 \dots$ 
   y  $\leftarrow 0.0$ 
   z  $\leftarrow \textit{angle}$ 
10:  // 24 itérations CORDIC i  $\leftarrow 0$ 
11:  while i < 24 do
   xOld  $\leftarrow x$ 
   pow2i  $\leftarrow$  table._getPow2(i)
12:   if z  $\geq 0$  then
   x  $\leftarrow x - y \times \textit{pow2i}$ 
   y  $\leftarrow y + xOld \times \textit{pow2i}$ 
   z  $\leftarrow z -$  table._getCordic(i)
13:   else
   x  $\leftarrow x + y \times \textit{pow2i}$ 
   y  $\leftarrow y - xOld \times \textit{pow2i}$ 
   z  $\leftarrow z +$  table._getCordic(i)
14:   end if
   i  $\leftarrow i + 1$ 
15: end while
16:  // Résultat final
17:  if inverse then
18:    return x ▷ Pour _cordicCos : retourner y
19:  else
20:    return y ▷ Pour _cordicCos : retourner x
21:  end if
22: end procedure
```

5.2.4 Implémentation des tables pré-calculées de cos et sin

- $CACHE_STEP = PI_OVER_2 / (CACHE_SIZE - 1)$
- $CACH_SIZE = 512$

Algorithm 7 Interpolation depuis la table sin

```
1: procedure _TABLESIN( $x$ )
2:   TableSinCos table = new TableSinCos();
   // instantie l'objet table de la classe TableSinCos (contenant les deux methodes
   _getSinValue( $i$ ) qui return  $\sin(i * CACHE\_STEP)$  et _getCosValue( $i$ ) qui return le
    $\cos(i * CACHE\_STEP)$ 
3:   // Calcul de l'indice dans la table
    $index\_f \leftarrow x / CACHE\_STEP$ 
    $i \leftarrow \lfloor index\_f \rfloor$ 
4:   if  $i \geq CACHE\_SIZE - 1$  then
5:     return table._getSinValue( $CACHE\_SIZE - 1$ )
6:   end if
7:   // Distance au point de table
    $h \leftarrow x - i \times CACHE\_STEP$ 
8:   // Approximation de Taylor pour  $h$  petit
    $\sin\_h \leftarrow h$ 
    $\cos\_h \leftarrow 1.0 - 0.5 \times h^2$ 
9:   // Formule d'addition :  $\sin(x) = \cos(h) \sin(i) + \sin(h) \cos(i)$ 
10:  return  $\cos\_h \times \text{table}.\_getSinValue(i) + \sin\_h \times \text{table}.\_getCosValue(i)$ 
11: end procedure
```

Algorithm 8 Interpolation depuis la table cos

```
1: procedure _TABLECOS( $x$ )
2:   // Même initialisation que _tableSin
3:   if  $i \geq CACHE\_SIZE - 1$  then
4:     return table._getCosValue( $CACHE\_SIZE - 1$ )
5:   end if
    $h \leftarrow x - i \times CACHE\_STEP$ 
    $\sin\_h \leftarrow h$ 
    $\cos\_h \leftarrow 1.0 - h^2 \times 0.5$ 
6:   // DIFFÉRENCE :  $\cos(x) = \cos(h) \cos(i) - \sin(h) \sin(i)$ 
7:   return  $\cos\_h \times \text{table}.\_getCosValue(i) - \sin\_h \times \text{table}.\_getSinValue(i)$ 
8: end procedure
```

5.2.5 Implémentation de la fonction cos

Algorithm 9 Implémentation de la fonction cos basé sur la fonction sin

```
1: procedure COS( $f$ )
2:   // Utilisation de l'identité  $\cos(x) = \sin(\pi/2 - x)$ 
3:   return  $\sin(\pi/2 - f)$ 
4: end procedure
```

5.3 Fonction atan

5.3.1 Implementation de la fonction atan

Algorithm 10 Fonction atan(x)

```

1: procedure ATAN( $f$ )
2:   // Gestion du signe  $negative \leftarrow false$   $x \leftarrow f$ 
3:   if  $x < 0$  then
4:      $negative \leftarrow true$ 
5:      $x \leftarrow -x$ 
6:   end if
7:   // Petites valeurs : série de Taylor (ordre 7)
8:   if  $x < 0.151$  then  $val \leftarrow x - x^3/3 + x^5/5 - x^7/7$ 
9:     if  $negative$  then return  $-val$ 
10:    elsereturn  $val$ 
11:    end if
12:  end if
13:  // Réduction d'intervalle  $[0, 1]$  pour CORDIC  $inverted \leftarrow false$ 
14:  if  $x > 1.0$  then
15:     $x \leftarrow 1.0/x$   $\triangleright \arctan(x) = \pi/2 - \arctan(1/x)$ 
16:     $inverted \leftarrow true$ 
17:  end if
18:  // CORDIC mode vectorisation  $curr\_x \leftarrow 1.0$   $curr\_y \leftarrow x$   $angle\_accumulee \leftarrow 0.0$ 
   $i \leftarrow 0$ 
19:  while  $i < 30$  do  $\triangleright$  Compromis précision/complexité  $x\_old \leftarrow curr\_x$ 
   $pow2i \leftarrow \text{table.}\_getPow2(i)$ 
20:    if  $curr\_y > 0$  then  $curr\_x \leftarrow curr\_x + curr\_y \times pow2i$   $curr\_y \leftarrow curr\_y -$ 
   $x\_old \times pow2i$   $angle\_accumulee \leftarrow angle\_accumulee + \text{table.}\_getCordic(i)$ 
21:    else  $curr\_x \leftarrow curr\_x - curr\_y \times pow2i$   $curr\_y \leftarrow curr\_y + x\_old \times pow2i$ 
   $angle\_accumulee \leftarrow angle\_accumulee - \text{table.}\_getCordic(i)$ 
22:    end if  $i \leftarrow i + 1$ 
23:  end while
24:  // Retransformation  $res \leftarrow angle\_accumulee$ 
25:  if  $inverted$  then  $res \leftarrow \pi/2 - res$ 
26:  end if
27:  if  $negative$  then  $res \leftarrow -res$ 
28:  end if
29:  return  $res$ 
30: end procedure

```

5.3.2 Réduction de l'argument pour la fonction Atan

$$\arctan(x) = \begin{cases} \arctan(1/x) - \pi/2 & x > 1 \\ -\arctan(1/x) + \pi/2 & x < -1 \\ \text{approximation Taylor} & |x| \leq 0.151 \\ \text{cordic vectoriel} & \text{sinon} \end{cases}$$

5.4 Fonction asin

Algorithm 11 Fonction asin(x)

```
1: procedure ASIN( $f$ )
2:   // Vérification du domaine  $[-1, 1]$ 
3:   if ( $f < -1.0 \parallel f > 1.0$ ) then( $f$ )
4:     return erreur
5:   end if
6:   // Cas particuliers près de  $\pm 1$   $\epsilon \leftarrow 10^{-7}$ 
7:   if  $|f - 1.0| \leq \epsilon$  then return  $\pi/2$ 
8:   end if
9:   if  $|f + 1.0| \leq \epsilon$  then return  $-\pi/2$ 
10:  end if
11:  if  $|f| > 0.9999$  then           ▷ Près de 1 mais différent  $\delta \leftarrow 1.0 - |f|$   $result \leftarrow \sqrt{2.0 \times \delta}$ 
12:    if  $f > 0$  then return  $\pi/2 - result$ 
13:    elsereturn  $result - \pi/2$ 
14:    end if
15:  end if
16:  // Plusieurs méthodes possibles pour  $\arcsin(x) = \arctan\left(\frac{x}{\sqrt{1-x^2}}\right)$ 
17:  Méthode choisie : Calcul de  $\sqrt{1-x^2}$  puis division  $x2 \leftarrow f \times f$   $denom \leftarrow \sqrt{1.0 - x2}$ 
18:  return  $\text{atan}(f/denom)$ 
```

▷ On a testé autres méthodes :

▷ 1. Avec $\sqrt{1-x^2}$ algo rapid (necessite gestion directe des bits

▷ 2. Avec racine inverse directe(moins performant

```
19: end procedure
```

5.5 Fonction acos

Algorithm 12 Fonction acos(x)

```
1: procedure ACOS( $f$ )
2:   // Utilisation de l'identité :  $\arccos(x) = \pi/2 - \arcsin(x)$   $res \leftarrow \text{asin}(f)$ 
3:   return  $\pi/2 - res$ 
4: end procedure
```

=====

=====

6 Validation et tests

6.1 Notre Stratégie de validation

1. Des fichiers .java sont creer pour remplir des fichiers .txt , sous la forme (nom_fichier ;fonction ;valeur ;resultat_attendu) (chaque ligne $\sim$ un test)
2. Ces fichiers .txt sont nommees comme suit , (par exemple TestsCos.txt , TestsSin.txt , TestsAtan.txt ... dans src/test/deca/Trigo/)
3. script creerTests.sh qui creer les tests dans deux repertoires valid et invalid , (si le resultat attendu est "erreur" , le tests est mis dans ./invalid/) **pour les tests de valid , on ajoute aussi un test binome pour chacun , ce nouveau test calcul la valeur ulp(x)**

4. en suite , un autre script `genererAss.sh` genere les executables `.ass`
5. en fin un script `verify_valid.sh` parcourt les tests dans `valid` (ne touche pas au binomes `_ulp`) et les executent , et compare leurs resultats avec le resultat attendu , et apres calcul le `ulp(x)` en executant le test binome `_ulp`
6. De meme un script `verify_invalid.sh` parcourt les tests dans `./invalid/` et verify qu'ils donnent erreur .

6.2 Genre de tests

- Des valeurs aléatoires distribuées dans le domaine
- Des Cas particuliers
- Des Cas Extremes , (comme 1 pour `asin`, `acos`) , (100000000 pour `sin/cos`)

6.3 Référence de précision et Visualisation des erreurs

Utilisation de `java.lang.Math` comme référence. et pour des valeurs connues ,comme `sin(0)=0` , `atan(0)=0` , on les mis directement (sans passer par java)

```
Test: ulp_b_3.14159
  Obtenu   : 2.38419e-07
  Attendu  : 2.384186e-07
  ULP      : 2.38419e-07
  Erreur relative : 0.000002 ULP
  OK ( $\leq 2$  ULP)

Test: ulp_b_3.4028236E33
  Obtenu   : 3.09485e+26
  Attendu  : 3.094850e+26
  ULP      : 3.09485e+26
  Erreur relative : 0.000000 ULP
  OK ( $\leq 2$  ULP)

Test: ulp_b_4.712388
  Obtenu   : 4.76837e-07
  Attendu  : 4.768372e-07
  ULP      : 4.76837e-07
  Erreur relative : 0.000000 ULP
  OK ( $\leq 2$  ULP)

Test: ulp_b_5.9
  Obtenu   : 4.76837e-07
  Attendu  : 4.768372e-07
  ULP      : 4.76837e-07
  Erreur relative : 0.000000 ULP
  OK ( $\leq 2$  ULP)

Test: ulp_b_6.28318
  Obtenu   : 4.76837e-07
  Attendu  : 4.768372e-07
  ULP      : 4.76837e-07
  Erreur relative : 0.000000 ULP
  OK ( $\leq 2$  ULP)
```

FIGURE 3 – ulp performance

```
Test: cos_314.15927
  Obtenu   : 2.32458e-05
  Attendu  : 1.000000e+00
  ULP      : 3.05176e-05
  Erreur relative : 32767.214794 ULP
  trop mal ( > 100 ULP)

Test: cos_315.73007
  Obtenu   : -2.76566e-05
  Attendu  : 0.000000e+00
  ULP      : 3.05176e-05
  Erreur relative : 0.906251 ULP
  OK (≤ 2 ULP)

Test: cos_4.1887903
  Obtenu   : 1.04720e+00
  Attendu  : -5.000000e-01
  ULP      : 4.76837e-07
  Erreur relative : 3244714.650918 ULP
  trop mal ( > 100 ULP)

Test: cos_4.712388
  Obtenu   : 1.57080e+00
  Attendu  : -9.417494e-07
  ULP      : 4.76837e-07
  Erreur relative : 3294209.429531 ULP
  trop mal ( > 100 ULP)

Test: cos_4.712389
  Obtenu   : 1.57080e+00
  Attendu  : 0.000000e+00
  ULP      : 4.76837e-07
  Erreur relative : 3294207.454539 ULP
  trop mal ( > 100 ULP)
```

FIGURE 4 – cos performance


```

1.07586... at source line 1
Test: asin_0.88
  Obtenu   : 1.07586e+00
  Attendu  : 1.075862e+00
1.075862e+00
  ULP      : 5.96046e-08
  Erreur relative : ULP
  OK ( $\leq 2$  ULP)

Test: asin_0.9
  Obtenu   : 1.11977e+00
  Attendu  : 1.119769e+00
  ULP      : 5.96046e-08
  Erreur relative : 16.777229 ULP
  un peu mal (>4 ULP et <100 ULP)

Test: asin_0.9659258
  Obtenu   : 1.30900e+00
  Attendu  : 1.308997e+00
  ULP      : 5.96046e-08
  Erreur relative : 50.331686 ULP
  un peu mal (>4 ULP et <100 ULP)

Test: asin_0.99
  Obtenu   : 1.42926e+00
  Attendu  : 1.429257e+00
  ULP      : 5.96046e-08
  Erreur relative : 50.331686 ULP
  un peu mal (>4 ULP et <100 ULP)

```

FIGURE 5 – atan performance

```

1.07586... at source line 1
Test: asin_0.88
  Obtenu   : 1.07586e+00
  Attendu  : 1.075862e+00
1.075862e+00
  ULP      : 5.96046e-08
  Erreur relative : ULP
  OK ( $\leq 2$  ULP)

Test: asin_0.9
  Obtenu   : 1.11977e+00
  Attendu  : 1.119769e+00
  ULP      : 5.96046e-08
  Erreur relative : 16.777229 ULP
  un peu mal (>4 ULP et <100 ULP)

Test: asin_0.9659258
  Obtenu   : 1.30900e+00
  Attendu  : 1.308997e+00
  ULP      : 5.96046e-08
  Erreur relative : 50.331686 ULP
  un peu mal (>4 ULP et <100 ULP)

Test: asin_0.99
  Obtenu   : 1.42926e+00
  Attendu  : 1.429257e+00
  ULP      : 5.96046e-08
  Erreur relative : 50.331686 ULP
  un peu mal (>4 ULP et <100 ULP)

```

FIGURE 6 – asin performance

6.4 Tests de performance

6.4.1 Temps d'exécution

Mesuré en cycles IMA pour différentes entrées :

Fonction	Cycles moyens	Variation
<code>sin</code>	140	± 20
<code>cos</code>	140	± 10
<code>asin</code>	200	± 20
<code>atan</code>	170	± 20
<code>atan</code>	100	± 10

TABLE 2 – Performance des fonctions

7 Limitations et perspectives

7.1 Limitations actuelles

7.1.1 Précision

- Erreur près des points de branchement (ex : `asin(1.0)`)
- Dégradation pour les très grands arguments pour `cos` et `sin` (réduction d'argument)

7.1.2 Performance

- Consommation mémoire des tables `Cos` et `Sin` et de `Cordic` (même avec just 5 niveaux)

7.2 Améliorations possibles

7.2.1 Algorithmiques

- Implémentation de la méthode complète de Payne-Hanek
- Approximations de minimax ou chebychev de degré plus élevé

7.3 Des Extensions possibles pour le futur

- Ajout de `tan`, `sinh`, `cosh`
- Fonctions hyperboliques inverses
- Versions en double précision

8 Conclusion

8.1 Récapitulatif des contributions

1. Conception et implémentation de l'extension `TRIGO`
2. Algorithmes optimisés pour la précision et la performance
3. Etudes Théoriques des Algorithmes et leurs limites et complexités
4. Analyse des compromis précision/énergie

Ce travail nous a permis de développer des compétences en :

- Analyse numérique et calcul flottant
- Optimisation d'algorithmes sous contraintes
- Génie logiciel appliqué aux systèmes embarqués (`CORDIC` , Gestion de Mémoire, `IMA..`)

Références

- [1] Article de synthèse pour L'algorithme CORDIC
- [2] Fast and accurate iterative generation of sine and cosine for equally-spaced angles
- [3] <https://www.infinimath.com/espaceeducation/tangenteeducation/articles/TE15/Cordic.pdf> les Secrets d'algo CORDIC
- [4] <https://www.bibmath.net/dico/index.php?action=affiche&quoi=.w/weierstrass.html> Idee sur l'approximation des fonctions par des polynomes (Théorme de Weierstrass)
- [5] <https://ens-lyon.hal.science/ensl-01529804v1/document> une these de doctorat avec des algorithmes utiles en double précision) je l'ai pas utilisé à cause de la representation IEEE 32bits ,mais c'était interessante
- [6] Les circuits de calcul trigonométriques, Wikilivres, *Fonctionnement d'un ordinateur*.
- [7] https://en.wikipedia.org/wiki/Minimax_approximation_algorithmAlgorithme Minimax
- [8] Cours de Modélisation Geometrique de l'ENSIMAG 2eme annee MMIS : base solid sur la gestion des polynomes pour faire le design des fonctions
- [9] Cours de Methode Numerique de l'ENSIMAG 1er annee : base solid sur la notion de conver=ce des Approximations et les suites (comme la suite de Cordic)